

Project Proposal Form, Fall 2026
Institute of Information Technology (IIT)
University of Dhaka

Student's Name:	Md. Habibur Rahman		
Student's Roll:	2507220	Phone:	01716115454

Project Title:

Design, Development and Evaluation of a Specification-Driven, Reusable Alumni Management Platform for Resource-Constrained Institutions

Motivation:

Alumni associations often depend on paper records, spreadsheets, informal messaging groups and manually maintained financial records. These practices make membership verification, payment tracking, communication, event coordination and governance-document management difficult.

The Govt. Haraganga College Alumni Association needs a low-cost, maintainable and privacy-aware system of record that volunteers can operate. The platform must work without enterprise-level staffing, recurring software licenses or live payment-gateway credentials.

This project focuses on reliable membership records, financial transparency, privacy-aware directory access, event and communication management, governance documents, mobile-first access, single-maintainer sustainability, specification-driven development and measurable testing. It also examines how far Association rules can be represented in software without allowing software to replace the authority of the constitution or Election Commission.

This is not only a CRUD application. Its research and engineering focus is the combination of constitutional rules represented as traceable and testable business rules, manual evidence-based payment as a deliberate primary workflow, a reusable and configurable platform foundation, operation by one maintainer at a low annual cost, portability across supported database providers and container hosts, safe handling of legacy databases and archived records, and quality evaluation against functional and non-functional requirements.

Project Description:

GHCAA is a multi-client platform consisting of an ASP.NET Core Web API (Clean Architecture), an Angular web application and a Flutter mobile application, with PostgreSQL for deployment and SQLite for local development and tests.

The project follows **specification-driven development**. Requirements are derived from stakeholder needs, current manual processes, Association documents and software-engineering standards, and each important requirement is traced to its source, business rule, implementation component, automated test and evaluation result.

Market and research position

The project was positioned against institutional advancement platforms, CiviCRM and other open-source membership systems, general-purpose CRM products such as Salesforce Nonprofit Success Pack, Zoho CRM and HubSpot, and community products such as Discourse. Those products are stronger in some mature functions, especially campaigns, reporting, directories, events and mentoring. The identified gap is their combination of:

- constitutional governance, auditable rules;
- manual payment with proof upload and officer verification as the primary path;
- Bengali/local identity and financial-channel support;
- data sovereignty and clean exit;
- mobile fitness;
- operation at a low annual cost.

The contribution is the combination and the documented trade-offs, not a claim that every individual feature is novel.

The main functional areas are:

membership and identity (registration, approval, digital ID, certificates, profiles, privacy controls, searchable directory);

accounting and payments (fees, dues, donations, configurable payment instructions, manual payment proof, administrative verification, receipts and audit history);

events and participation (publication, registration, capacity control, waitlisting, QR attendance, budgets and moderated photo submissions);

communication and networking (news, notices, targeted communication, real-time messaging, notifications, jobs and mentorship); and

governance and administration (constitution versioning, Executive Committee records, amendment voting, polls, voter-roll preparation and election-document support).

The platform supports governance procedures but does not replace the Election Commission or claims to provide a legally sufficient **electronic election system**.

Portability, legacy data and configuration

The persistence boundary is provider-neutral to the extent required by the specification: PostgreSQL is the deployment provider, and SQLite is used for local development and tests without changing business source code. The deployment is also container-portable and is intended to run on a container host without a proprietary platform dependency.

Legacy handling is part of the design rather than an afterthought. Migration bootstrap detects databases created by the earlier schema mechanism, baselines existing objects when appropriate, apply only genuinely pending migrations, and logs failures. Membership and financial records use archival deletion and query filters, so ordinary screens hide archived rows without destroying audit evidence. Full export in an open format is a portability requirement.

The configuration model has two planes:

- **Runtime configuration:** organization name, branding, contact details, currency, feature flags, workflow settings, membership presentation, payment instructions and public content can be changed by authorized administrators without redeployment.
- **Boot profile configuration:** a profile pack supplies fresh-deployment defaults, seed data, assets, documents, email templates, lookups, membership tiers and SEO information through `ORG_PROFILE`.

The profile-pack goal is deployment of a new organization from configuration and assets rather than a source fork. The current prototype already provides substantial runtime and profile support, while the remaining brand-literal cleanup and full zero-source-change onboarding remain explicit implementation work. This distinction prevents a planned white-label capability from being presented as fully proven.

Reusable platform foundation

The platform is **built as a reusable foundation** rather than a one-off application. Authentication, role-based authorization, accounting and ledger services, audit logging, file handling, SignalR notifications, organization configuration, shared validation, reusable Angular controls (Shared frontend controls include page headers, search fields, filters, data tables, pagination, loading states, empty states, confirmation dialogs and shared form patterns) and Flutter widgets, and CI/CD are designed as organization-agnostic services, separate from Association-specific rules and configuration. Runtime settings such as branding, currency, feature flags and payment instructions can be changed by administrators without redeployment, and a boot profile pack can seed a fresh deployment's defaults, documents and content.

Non-functional requirements cover performance, security, reliability, usability, maintainability, portability, recovery, auditability and privacy, evaluated through quality-attribute scenarios such as directory latency, session

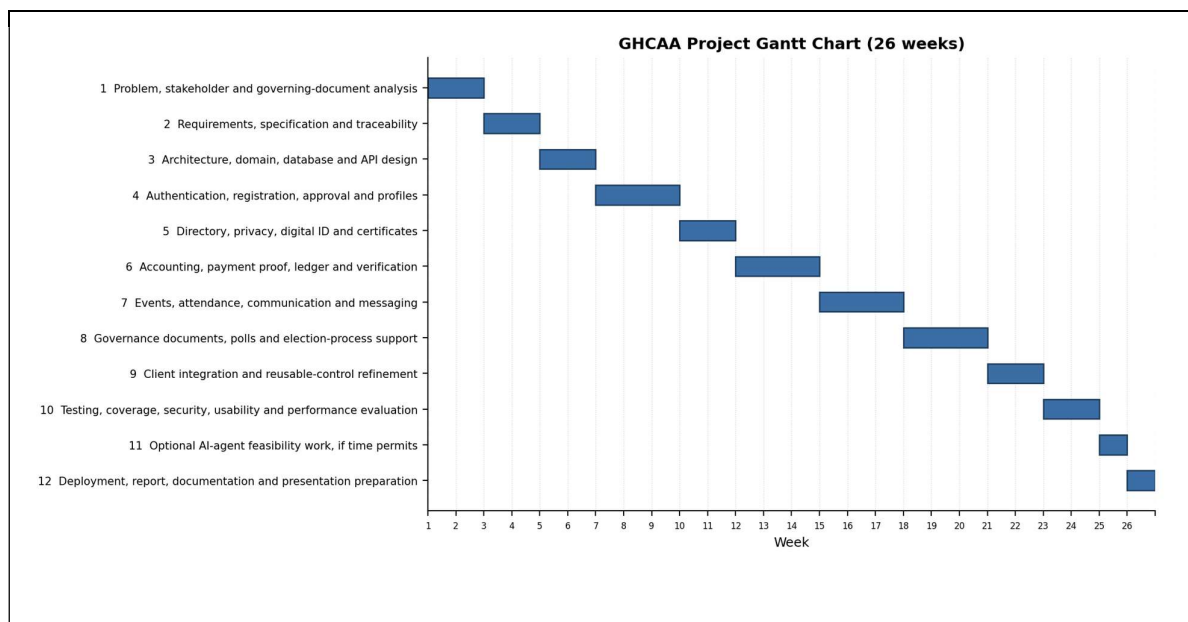
invalidation on status change, dependency-failure isolation, architecture-rule enforcement and recovery time. The prototype already includes the backend, web app, mobile app, authentication, accounting, membership, events, governance and CI/CD. Remaining work covers traceability closure, measured coverage, security and usability evidence, performance evaluation and documentation.

Strategy & Timeline:

The project Gantt chart is given below:

Week	Task No / Module	Description	Dependencies
1	Problem & stakeholder analysis	Stakeholder needs, current manual processes and governing documents (Membership, constitution, election rules, manual forms) reviewed and analyzed.	—
2	Stakeholder and Document Review	Governing-document analysis completed; Review stakeholder needs, problem statement and scope finalized.	1
3	Requirements & specification	Define functional requirements, quality requirements, scope and limitations.	1-2
4	Traceability and Acceptance Criteria	Requirement traceability (source requirements -> use cases -> business rule -> component -> test) drafted for core modules.	3
5	Domain and Data Modelling	Model members, payments, events, governance records, users and related entities.	4
6	Architecture and API Design	Finalize Clean Architecture, API contracts, database design and client integration plan.	5
7	Authentication	Implement registration, login, password security, JWT sessions and account recovery flows.	6
8	Authorization and User Roles	Implement role-based access control, protected routes and administrative permissions.	7
9	Membership Management	Implement registration, academic records, professional records, approval and membership status.	7–8
10	Profile and Privacy	Implement profiles, privacy controls, secure documents and member preferences.	9
11	Directory and Digital Identity	Implement alumni search, privacy-aware directory results, digital ID cards and certificates.	10
12	Accounting Foundation	Implement fees, dues, donations, income, expenses and ledger structures.	6, 9

13	Manual Payment Workflow	Implement payment instructions, proof upload, administrative verification and payment status.	12
14	Financial Audit and Receipts	Implement audit history, financial summaries and downloadable payment receipts.	13
15	Event Management	Implement event creation, publication, registration, capacity and waitlisting.	9
16	Attendance and Gallery	Implement QR attendance, event tasks, budgets, expenses and moderated gallery submissions.	15
17	Communication and Messaging	Implement news, notices, notifications, targeted communication and real-time messaging.	8, 15
18	Governance Documents	Implement constitution publication, version history and Executive Committee records.	6, 9
19	Voting and Polls	Implement amendment voting, non-binding polls and eligibility controls.	18
20	Election Support	Implement voter-roll preparation, candidate information and election-document support.	18–19
21	Reusable Platform Refinement	Consolidate authentication, authorization, accounting, shared controls, organization configuration and common services.	7–20
22	Web and Mobile Integration	Complete Angular and Flutter integration against the shared API contracts.	21
23	Automated Testing	Run backend, web, mobile, API, integration and end-to-end tests. Add missing critical tests.	22
24	Coverage and Quality Evaluation	Collect coverage, requirement coverage, static-analysis results and contract-test evidence.	23
25	Security, Usability and Performance	Perform security review, privacy checks, usability evaluation and performance measurements.	23–24
26	Deployment and Final Documentation	Validate CI/CD, Docker packaging and deployment; complete report, presentation and future-work documentation.	25
26	AI-Agent Feasibility Extension (Optional)	Add a controlled, advisory AI agent for policy lookup, portal navigation and approved information retrieval. The agent will use existing authentication, authorization and privacy controls and will not approve membership, verify payments, modify accounting records or determine election eligibility.	1-20



Languages or Tools to be used:

- **Frontend:** Angular, TypeScript, HTML, SCSS, Angular Signals
- **Backend:** ASP.NET Core 9 Web API, C#, Clean Architecture, PostgreSQL (deployment) / SQLite (local & tests)
- **Database:** PostgreSQL (deployment), SQLite (local development and tests)
- **IDE / Editor:** Visual Studio Code, MS Visual Studio, Antigravity
- **AI Tools / Assistants (if any)** — e.g., **GitHub Copilot, ChatGPT, Claude:** GitHub Copilot (via Copilot SDK in VS Code), Claude
- **Mobile:** Flutter and Dart
- **Real-time:** ASP.NET Core SignalR
- **Authentication:** JWT, BCrypt and role-based authorization
- **Testing:** NUnit, Moq, FluentAssertions, Vitest, Playwright, Flutter Test and integration tests
- **Coverage:** Coverlet and .NET test tooling
- **Quality checks:** .NET formatting, TypeScript type checking, Dart analysis and CI validation
- **Version control:** Git and GitHub
- **CI/CD:** GitHub Actions
- **Packaging and deployment:** Docker and hosted pre-production deployment

AI Tool Use Declaration:

- ☒ I intend to use AI coding/writing assistance tools for this project.
- ☐ I do not intend to use AI tools for this project.

If yes, tool(s) to be used: GitHub Copilot (Copilot SDK, VS Code), Claude.

Intended purpose(s) (e.g., boilerplate generation, debugging, documentation drafting, learning a new framework): Framework/library learning, boilerplate generation, debugging, test suggestions, code review, documentation drafting, technical comparison and review implementation decisions.

All AI-assisted output is reviewed, tested and modified by the student, who remains responsible for requirements, architecture, implementation, testing and security decisions.

Declaration: I understand that, regardless of AI tool use, I am required to disclose such use in my project report, and I must be able to independently explain, defend, and modify any part of my code — including any AI-

assisted sections — during the midterm and final presentations, live defense, and supervisor meetings throughout the semester. I understand that submitting AI-generated work I cannot explain or defend may result in a reduced grade or other academic consequences.

Student's Signature: _____

Date: _____

Project GitHub link: <https://github.com/shalin-rahman/GHCAA>

Note: Maintain a regular, incremental, and dated commit history throughout the semester. A single large commit submitted close to the deadline is treated as a red flag and may be investigated.

Supervisor's Name: _____

Signature of the supervisor: _____

Date: _____

Before the Midterm Presentation:

I confirm that the progress is satisfactory and I am forwarding it for the midterm presentation.

I further confirm that the student has demonstrated independent understanding of the codebase — including any AI-assisted portions — and was able to explain and modify their prototype during our supervision meetings.

Signature of the supervisor: _____

Date: _____

Before the Final Presentation:

I confirm that the progress is satisfactory and I am forwarding it for the final presentation.

I further confirm that the student has consistently demonstrated understanding and ownership of their work — including any AI-assisted code — throughout the semester, based on regular supervision meetings and progress reviews.

Signature of the supervisor: _____

Date: _____

Proposal Presentation Feedback (if any):

Midterm Presentation Feedback (if any):

AI-Use Disclosure Review Notes (if any):

(For supervisor/evaluator use — note any discrepancy between disclosed AI use and demonstrated understanding.)